

Predicting 30-Day Hospital Readmission in Diabetic Patients (Part 1)

Week 2 Project Report

Prepared by: Peter Adepoju

Email: peter.o.adepoju@gmail.com

Dataset: UCI Diabetes 130-US Hospitals

Date: June 12, 2026

Contents

1	Introduction	3
1.1	The Hospital Readmission Problem	3
1.2	Diabetes as a High-Risk Condition	4
1.3	The CMS Hospital Readmissions Reduction Program	4
1.4	The Role of Machine Learning in Clinical Decision Support	5
1.5	Research Questions	5
2	Dataset Description	6
2.1	The UCI Diabetes 130-US Hospitals Dataset	6
2.2	Raw Dataset Structure	7
2.3	The ‘?’ Missing Value Encoding	7
2.4	The Target Variable: <code>readmitted</code>	8
2.5	Patient-Level vs. Encounter-Level Data	9
2.6	Raw Dataset Summary Statistics	10
3	Data Cleaning	11
3.1	Step 1 — Replacing ‘?’ with NaN	11
3.2	Step 2 — Column Exclusion Decisions	11
3.3	Step 3 — Clinical Row Exclusions	12
3.4	Step 4 — Gender Handling	13
3.5	Step 5 — Deduplication: First-Encounter Strategy	13
3.6	Step 6 — Binary Target Construction	13
3.7	Step 7 — Handling Remaining Diagnosis Missingness	14
3.8	Cleaning Audit: Before and After	14
4	Feature Engineering	15
4.1	The Engineering Philosophy	15
4.2	ICD-9 Diagnosis Code Grouping	15
4.3	Age Ordinal Encoding	16
4.4	The 24 Medication Columns	17

4.5	Hospital Utilisation Features	18
4.6	A1C Result and Glucose Serum Encoding	18
4.7	Binary Clinical Flags	19
4.8	Gender Binary Encoding	19
4.9	Final Feature Space	19
References		21

Chapter 1

Introduction

“The goal of medicine is to cure sometimes, to relieve often, and to comfort always.”

— Hippocrates

1.1. The Hospital Readmission Problem

When a patient is discharged from hospital, the expectation on both sides is that the acute episode has been adequately managed. A readmission within 30 days (meaning a return to inpatient care for a potentially related condition) signals that this expectation was not met. The 30-day window reflects the period during which post-discharge complications, inadequate follow-up, medication errors, and care coordination failures are most likely to surface. Beyond this window, readmissions are considered less attributable to the index hospitalisation.

The scale of the problem in the United States is substantial. Approximately **20%** of Medicare patients are readmitted within 30 days of discharge, generating roughly **\$26 billion** in additional expenditure annually (Centers for Medicare & Medicaid Services, 2013).¹

van Walraven, Bennett, Jennings, Austin, and Forster (2011) estimated that between 9% and 59% of readmissions are potentially preventable, depending on the clinical condition and the review criteria applied. Even conservative estimates imply that tens of thousands of hospitalisations per year could be avoided with better post-discharge care.

Each readmission carries its own risk of hospital-acquired infections, venous thromboembolism, procedural complications, and the general morbidity associated with being acutely unwell. For elderly patients with multiple comorbidities (the dominant demographic in this dataset) each hospitalisation accelerates functional decline and reduces quality of life.

¹The MedPAC (Medicare Payment Advisory Commission) produces annual reports on readmission trends and costs. See <https://www.medpac.gov> for the most recent statistics.

1.2. Diabetes as a High-Risk Condition

Not all medical conditions carry equal readmission risk. Diabetes mellitus is one of the conditions most strongly associated with repeated hospitalisation, for several mutually reinforcing reasons.

1. **Medication complexity.** Diabetic management often involves multiple medications such as insulin, oral hypoglycaemics, antihypertensives, statins and requires careful titration. Errors in dosing after discharge are common and can precipitate acute crises.
2. **Glycaemic instability.** Hospitalisation itself disrupts glycaemic control through stress responses, changes in diet, and altered medication schedules. Patients discharged in an unstable glycaemic state face elevated readmission risk.
3. **Comorbidity burden.** The dataset we analyse records a mean of 7.3 diagnoses per encounter, reflecting the reality that diabetic inpatients are rarely admitted for diabetes alone. Cardiovascular disease, renal impairment, and peripheral neuropathy are common companions.
4. **Social determinants.** Diabetes disproportionately affects lower-income and minority populations who may have limited access to primary care, nutritional resources, and medication adherence support after discharge.

The dataset we use in this study, the UCI Diabetes 130-US Hospitals dataset (Strack et al., 2014), was collected precisely because diabetic patients represent a high-stakes, high-cost, and potentially modifiable readmission cohort.

1.3. The CMS Hospital Readmissions Reduction Program

The policy context for readmission prediction changed fundamentally in 2012 with the introduction of the **Hospital Readmissions Reduction Program (HRRP)** under the Affordable Care Act. The HRRP authorises the Centers for Medicare & Medicaid Services (CMS) to impose financial penalties on hospitals with higher-than-expected 30-day readmission rates for specific conditions, initially including heart failure, pneumonia, and acute myocardial infarction, and later expanded to include total hip and knee arthroplasty and chronic obstructive pulmonary disease (Centers for Medicare & Medicaid Services, 2013).²

The maximum penalty has grown from 1% of base operating payments in fiscal year 2013 to 3% from fiscal year 2015 onwards. For large hospitals, this represents millions of dollars annually. The programme created immediate commercial demand for readmission prediction tools, and a substantial literature on predictive modelling for readmission emerged in the years following its introduction.

²Full technical details of the HRRP, including the excess readmission ratio calculation methodology, are documented at <https://www.cms.gov/medicare/payment/prospective-payment-systems/acute-inpatient-pps/readmissions-reduction-program-hrrp>.

Crucially, the HRRP uses risk-adjusted expected readmission rates, comparing a hospital's actual performance to what would be predicted given its patient population's risk factors. This risk-adjustment methodology is itself a machine learning problem, making the study of readmission prediction directly relevant to healthcare policy.

1.4. The Role of Machine Learning in Clinical Decision Support

The traditional approach to clinical risk stratification relies on hand-crafted scoring systems: the LACE Index (Length of stay, Acuity of admission, Comorbidities, Emergency department use), the HOSPITAL Score, and similar instruments (Donné, Aujesky, Williams, & Schnipper, 2013). These tools are simple to implement at the bedside but sacrifice predictive accuracy for interpretability.

Machine learning offers a different tradeoff: by learning non-linear interactions among dozens of clinical variables simultaneously, it can in principle capture patterns that simple additive risk scores miss. The question is whether the incremental predictive gain justifies the increased complexity and the risks of deploying an opaque model in a clinical setting.

The specific task in this project is **binary classification**: given the clinical record of a diabetic inpatient at the time of discharge, predict whether they will be readmitted within 30 days ($y = 1$) or not ($y = 0$). This is a supervised learning problem, and every major design decision in our pipeline: how we split the data, which metric we optimise, how we handle the class imbalance, is shaped by the clinical context.

1.5. Research Questions

This project is organised around five research questions:

1. Can a machine learning classifier trained on structured clinical discharge data predict 30-day readmission with ROC-AUC ≥ 0.75 in a held-out test set?
2. Do ensemble methods (Random Forest, Gradient Boosting, XGBoost) outperform Logistic Regression on this task, and by how much?
3. Which clinical features are the most important predictors of readmission?
4. At what classification threshold does the model provide the most clinically useful balance between precision (minimising unnecessary follow-up) and recall (minimising missed readmissions)?
5. Are there meaningful performance disparities across demographic subgroups (race, age, gender) that would raise fairness concerns in a deployment context?

Chapter 2

Dataset Description

2.1. The UCI Diabetes 130-US Hospitals Dataset

The primary data source for this project was originally collected and described by [Strack et al. \(2014\)](#) and deposited in the UCI Machine Learning Repository ([Dua & Graff, 2019](#)).¹ It represents one of the largest publicly available structured clinical datasets for the study of diabetes-related hospitalisation.

The dataset comprises **101,766 inpatient encounters** drawn from **130 US hospitals** over a ten-year period (1999–2008).

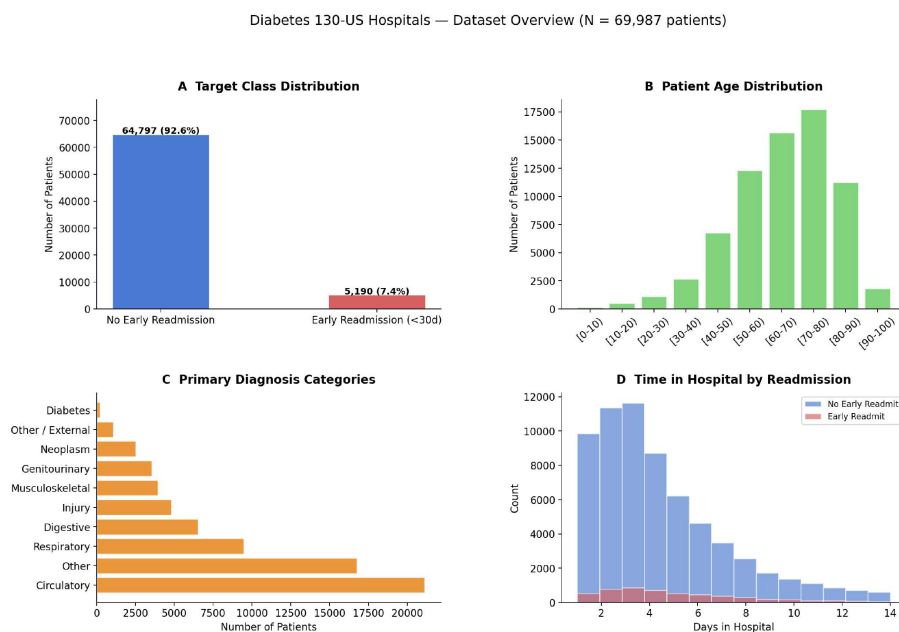


Figure 2.1. Overview of the UCI Diabetes 130-US Hospitals dataset, showing age distribution, class balance, admission type breakdown, and primary diagnosis category distribution. These four panels summarise the key population-level characteristics discussed in Sections 2.2–2.5.

¹The dataset is freely available at <https://archive.ics.uci.edu/ml/datasets/Diabetes+130-US+hospitals+for+years+1999-2008>.

Each row represents a single hospital encounter for a patient satisfying the following inclusion criteria at the time of the original study:

- The patient was diagnosed with diabetes (as a primary or secondary diagnosis).
- The length of stay was between 1 and 14 days.
- Laboratory tests were performed during the encounter.
- Medications were administered during the encounter.

These criteria mean the dataset captures a relatively complex inpatient cohort: patients who were unwell enough to require laboratory testing and medication management, but not so acutely ill as to be in intensive care units.

2.2. Raw Dataset Structure

The raw dataset contains **50 columns**. These fall into five functional categories, summarised in Table 2.1.

Table 2.1. Feature categories in the raw UCI dataset.

Category	Count	Examples
Patient identifiers	2	encounter_id, patient_nbr
Administrative	9	admission_type_id, payer_code, medical_specialty
Clinical measurements	9	time_in_hospital, num_lab_procedures, A1Cresult
Diabetes medications	24	metformin, insulin, glipizide, ...
ICD-9 diagnoses	3	diag_1, diag_2, diag_3
Target variable	1	readmitted
Demographic	2	race, gender

The 24 medication columns each record the change in dosage of a specific diabetes drug during the encounter, using one of four categorical values: **No** (not prescribed), **Steady** (dose unchanged), **Down** (dose decreased), or **Up** (dose increased). The complete list of drugs includes metformin, repaglinide, nateglinide, chlorpropamide, glimepiride, acetohexamide, glipizide, glyburide, tolbutamide, pioglitazone, rosiglitazone, acarbose, miglitol, troglitazone, tolazamide, examide, citoglipton, insulin, glyburide-metformin, glipizide-metformin, glimepiride-pioglitazone, metformin-rosiglitazone, metformin-pioglitazone, and insulin. This breadth reflects the complexity of diabetic pharmacotherapy in a multi-institution dataset spanning a decade.

2.3. The ‘?’ Missing Value Encoding

A critical data quality issue that must be addressed before any analysis is the dataset’s *non-standard missing value sentinel*. Many columns that contain missing data encode this missingness as the string ‘?’ rather than as a proper NaN or NULL value. This causes a subtle but serious problem: pandas’

default `read_csv()` will read these as the literal string “?”, and standard missingness checks such as `df.isnull()` or `df.isna()` will return `False` for these entries, silently treating the “?” as a valid category.²

The first step in any responsible analysis of this dataset is therefore to replace all occurrences of ‘?’ with `NaN` using `df.replace('?', np.nan, inplace=True)`. After this replacement, the columns with substantial missingness are:

Table 2.2. Missing value rates after ‘?’ replacement in the raw dataset (101,766 rows before cleaning).

Column	Missing Count	Missing %	Action Taken
weight	98,569	96.9%	Dropped
medical_specialty	49,949	49.1%	Dropped
payer_code	40,256	39.6%	Dropped
race	2,273	2.2%	Retained (mode imputation after clean)
diag_2	358	0.35%	Retained (filled with ‘Unknown’)
diag_3	1,423	1.4%	Retained (filled with ‘Unknown’)
diag_1	21	<0.1%	Retained (filled with ‘Unknown’)

The high missingness in `weight` (96.9%) makes it essentially unusable, imputation of this many missing values would introduce more noise than signal. `medical_specialty` and `payer_code` are also dropped.

2.4. The Target Variable: `readmitted`

The raw target column `readmitted` takes three values:

- `NO`: Not readmitted within the observation window.
- `>30`: Readmitted more than 30 days after discharge.
- `<30`: Readmitted within 30 days of discharge.

We collapse this to a binary target: $y = 1$ if `readmitted == '<30'`, else $y = 0$. This binarisation means that patients who are readmitted after 30 days are treated identically to patients who are never readmitted. The clinical justification is that a readmission after 30 days is less likely to be attributable to the index hospitalisation and is therefore outside the scope of the HRRP-motivated prediction task. The raw class distribution before deduplication is: `NO` = 54.0%, `>30` = 34.9%, `<30` = 11.1%.

²Pandas allows custom missing-value markers to be interpreted as `NaN` during data import using the `na_values` argument. See: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html.

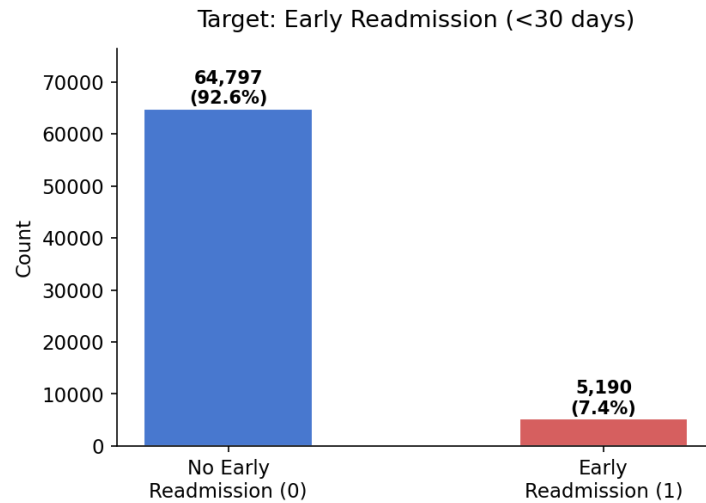


Figure 2.2. Distribution of the binary readmission label in the cleaned, deduplicated working cohort (69,987 encounters). The positive class (<30 days) comprises 11.2% of encounters, creating a substantial class imbalance that requires explicit methodological treatment.

2.5. Patient-Level vs. Encounter-Level Data

A structural feature of this dataset that has profound implications for modelling is that each row represents a *hospital encounter*, not a *unique patient*. The dataset contains 101,766 encounters for approximately **71,518 unique patients** (column: `patient_nbr`), meaning that many patients appear in multiple rows, sometimes with 5–10 encounters over the ten-year observation window.

This structure creates a risk of **patient-level data leakage**: if a patient’s encounters are split across training and test sets, the model can implicitly “learn” about this specific patient from their training encounters, and then appear to generalise well to their test encounters when it is actually memorising patient-specific patterns. This would produce optimistically biased performance estimates on what is effectively *in-distribution* data. The correct resolution is splitting by patient ID rather than by row.

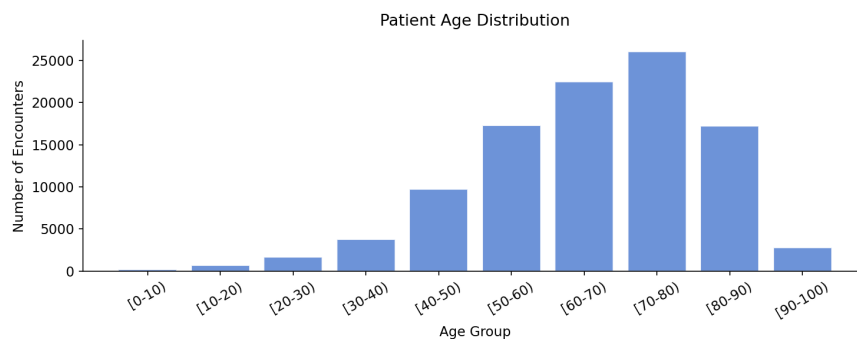


Figure 2.3. Distribution of patient age groups in the raw dataset (101,766 encounters). Ages are recorded in decade brackets (e.g. [70–80)). The distribution is skewed toward older patients, with the modal group being the 70–80 decade. This age profile is consistent with the epidemiology of type 2 diabetes in the US during the study period.

2.6. Raw Dataset Summary Statistics

Selected summary statistics for the key clinical features (before cleaning) are presented in Table 2.3. These figures establish the baseline picture of the patient population.

Table 2.3. Summary statistics for selected clinical variables across all 101,766 raw encounters. The high means and right-skewed distributions of prior visit counts foreshadow the need for log-transformation before modelling.

Feature	Mean	Std	Median	75th pct.	Max
Time in hospital (days)	4.29	2.94	3.0	6.0	14
Lab procedures	43.0	19.8	44.0	57.0	132
Procedures performed	1.40	1.74	1.0	2.0	6
Medications count	15.75	8.25	14.0	20.0	81
Distinct diagnoses	7.27	1.99	8.0	9.0	16
Prior outpatient visits	0.30	1.11	0.0	0.0	42
Prior emergency visits	0.12	0.63	0.0	0.0	54
Prior inpatient visits	0.33	0.82	0.0	0.0	17

Several features of this table deserve attention. First, `num_lab_procedures` has a mean of 43 and a maximum of 132, reflecting the intensive laboratory monitoring typical of diabetic inpatient care. Second, `number_diagnoses` has a mean of 7.27 and a minimum of 1, confirming the high comorbidity burden of this cohort. Third, all prior visit counts (`number_outpatient`, `number_emergency`, `number_inpatient`) have medians of 0 but long right tails with maximums in the tens to hundreds. These heavily right-skewed distributions will require log-transformation for linear models to function effectively.

Chapter 3

Data Cleaning

“Data scientists spend 80% of their time cleaning data and 20% complaining about it.”

— Attributed to various practitioners

Good data cleaning is the process of translating domain knowledge about *how the data was collected* into decisions about *what the data means*. Every step below is motivated by a specific clinical or statistical rationale.

3.1. Step 1 — Replacing ‘?’ with NaN

As discussed in Chapter 2, the dataset encodes missing values as the string ‘?’ in multiple columns. The first cleaning step replaces all such occurrences with `numpy.nan`:

```
1 df.replace('?', np.nan, inplace=True)
```

Listing 3.1. Replacing the ‘?’ sentinel with proper NaN values.

After this replacement, the complete inventory of missing values can be computed reliably with `df.isnull().sum()`, revealing the distribution shown in Table 2.2 (Chapter 2).

3.2. Step 2 — Column Exclusion Decisions

Four columns are excluded from the analysis before any modelling begins.

weight (96.9% missing)

With fewer than 4% of patients having a recorded weight, no imputation strategy can recover meaningful signal. Multiple imputation (Rubin, 1987) requires data that is *Missing at Random (MAR)*, a defensible assumption only when the proportion missing is moderate. At 96.9%, the missing weight data is overwhelmingly likely to be *Missing Not at Random (MNAR)*: weight is systematically not documented in

this dataset, probably because it was not routinely recorded in electronic health record systems during the 1999–2008 period. Imputing from such sparse data would introduce far more noise than information.¹

payer_code (39.6% missing)

This column records the insurance/payer type (Medicare, Medicaid, private, self-pay, etc.). Beyond its high missingness rate, this variable raises an important methodological concern: payer type is a proxy for socioeconomic status and is likely to be both highly correlated with race and gender (protected attributes) and subject to systematic collection bias across hospital systems. Including it risks amplifying socioeconomic disparities into the model’s predictions.

medical_specialty (49.1% missing)

This column records the admitting physician’s specialty. While clinically meaningful, its near-50% missingness makes it unreliable, and the specialties that *are* recorded may not be representative of the full cohort. Imputing the mode would mask true heterogeneity; dropping the column is more honest.

encounter_id

This is a pure database identifier with no clinical meaning. Including it would create a spurious predictor that perfectly identifies each row in the training data but generalises to nothing.

Columns retained despite high missingness

Notably, `A1Cresult` and `max_glu_serum` are retained despite substantial missingness. The clinical interpretation is that whether a physician *ordered* an HbA1c test or a serum glucose measurement is itself a signal: clinicians order these tests when they have clinical suspicion of poor glycaemic control. A missing value in these columns thus means “test not ordered,” which is clinically different from “test ordered but result unavailable.” Both columns are encoded to preserve this distinction.

3.3. Step 3 — Clinical Row Exclusions

Not all rows represent patients for whom 30-day readmission is a meaningful outcome. Some patients were discharged to post-acute facilities, transferred to other hospitals, or died during the index encounter. Predicting “readmission” for patients who died is logically incoherent, since a dead patient cannot be readmitted.

We exclude rows where `discharge_disposition_id` corresponds to patient death, hospice, or care facility transfer, specifically codes 11 (“Expired”), 13 (“Hospice/Medical Facility”), 14 (“Hospice/Home”), 19 (“Expired at Home”), 20 (“Expired in a Medical Facility”), and 21 (“Expired – Place Unknown”). These exclusions remove approximately 2,600 rows from the raw dataset.

¹The taxonomy of Missing Completely at Random (MCAR), Missing at Random (MAR), and Missing Not at Random (MNAR) is developed in detail by Rubin (1987). A practical tutorial is available at <https://stefvanbuuren.name/fimd/>, which is the companion website to a thorough treatment of missing data in practice.

3.4. Step 4 — Gender Handling

A small number of rows carry the value 'Unknown/Invalid' in the gender column. Given their minimal count (fewer than 5 rows), these are excluded rather than imputed. The clinical assumption is that the "Unknown/Invalid" label reflects a data entry error rather than a meaningful clinical category.

3.5. Step 5 — Deduplication: First-Encounter Strategy

As established in Section 2.5, the same patient may appear in multiple rows. We retain only the **first encounter per patient**, defined as the encounter with the lowest encounter ID (a proxy for chronological order within the dataset).

Several alternative deduplication strategies were considered:

- **Random encounter:** Introduces randomness into the cohort definition, reducing reproducibility.
- **Last encounter:** Would capture the patient's most recent clinical state, which is useful for longitudinal prognosis but not for the index-admission prediction task.
- **Most severe encounter:** Selecting the encounter with the most diagnoses or longest stay would bias the cohort toward extreme cases.
- **First encounter:** Models the prospective clinical scenario in which a model is deployed — at the *first* time a patient is admitted, can we predict whether they will be readmitted? This is the most realistic deployment context.

The first-encounter strategy also eliminates the patient-level leakage problem at its source: with one row per patient, there is no risk of training and test sets containing multiple encounters from the same patient.

Deduplication reduces the dataset from 101,766 rows to **69,987 rows**.

3.6. Step 6 — Binary Target Construction

After deduplication, the target variable is binarised:

```
1 df['readmitted_binary'] = (df['readmitted'] == '<30').astype(int)
2 df.drop(columns=['readmitted'], inplace=True)
```

Listing 3.2. Binarising the readmission target.

The resulting label distribution is **88.8%** negative (not readmitted within 30 days) and **11.2%** positive (readmitted within 30 days).

3.7. Step 7 — Handling Remaining Diagnosis Missingness

After the exclusion steps, a small number of rows have missing values in `diag_1`, `diag_2`, and `diag_3`. These are filled with the string 'Unknown' before the ICD-9 grouping step, which will assign them to an 'Other' diagnosis category. This is preferable to row-level exclusion (which would reduce sample size further) or mode imputation (which would artificially inflate the most common diagnosis category).

3.8. Cleaning Audit: Before and After

Table 3.1 summarises the before-and-after state of the dataset after all cleaning steps.

Table 3.1. Dataset dimensions before and after each cleaning step. The final working dataset retains 69,987 encounters across 45 columns with no remaining missing values.

Step	Operation	Rows	Columns
0	Raw dataset	101,766	50
1	Replace '?' with NaN	101,766	50
2	Drop <code>weight</code> , <code>payer_code</code> , <code>medical_specialty</code> , <code>encounter_id</code>	101,766	46
3	Remove deceased/hospice discharges	99,059	46
4	Remove invalid gender rows	99,006	46
5	First-encounter deduplication	69,987	46
6	Binarise target; drop raw column	69,987	46
7	Fill missing diagnoses with 'Unknown'	69,987	46
Final	Zero missing values	69,987	46

Chapter 4

Feature Engineering

Feature engineering is the process of transforming raw data representations into a numerical feature matrix that machine learning models can operate on effectively. It is simultaneously the most domain-knowledge-intensive and the most impactful phase of any ML pipeline. Poor feature engineering can prevent an excellent algorithm from discovering real patterns; good feature engineering can enable even simple models to perform competitively.

4.1. The Engineering Philosophy

The raw dataset is not directly usable by most ML algorithms. It contains string-valued categorical fields, decade-bracket age encodings, and diagnosis codes in ICD-9 format. The engineering steps below convert these into numerical representations while preserving as much clinically meaningful structure as possible. The guiding principle is *semantic fidelity*: the numerical encoding should preserve the meaningful relationships present in the raw data.¹

4.2. ICD-9 Diagnosis Code Grouping

The three diagnosis columns (`diag_1`, `diag_2`, `diag_3`) contain raw ICD-9 (International Classification of Diseases, 9th revision) codes.² There are thousands of distinct ICD-9 codes across the three columns. Using them as raw categorical features would create a massively sparse, high-dimensional representation with extremely high variance.

We instead apply a grouping function that maps each ICD-9 code to one of nine clinically meaningful categories:

¹A comprehensive treatment of feature engineering for structured (tabular) data is the book *Feature Engineering for Machine Learning* by Alice Zheng and Amanda Casari (O'Reilly, 2018), available at <https://www.oreilly.com/library/view/feature-engineering-for/9781491953235/>.

²The ICD-9 classification system is documented by the World Health Organization at <https://www.who.int/standards/classifications/classification-of-diseases>. It was the standard US clinical coding system until ICD-10 replaced it in October 2015 — after the end of this dataset's observation period.

Table 4.1. Mapping from ICD-9 code ranges to nine clinical categories.

Category	ICD-9 Code Range	Clinical Scope
Circulatory	390–459, 785	Heart disease, hypertension, stroke
Respiratory	460–519, 786	Pneumonia, COPD, asthma
Digestive	520–579, 787	GI disorders, liver disease
Diabetes	250.xx	All diabetes-specific codes
Injury	800–999	Trauma, fractures, poisoning
Musculoskeletal	710–739	Arthritis, orthopaedic conditions
Genitourinary	580–629, 788	Renal failure, UTI, renal disease
Neoplasm	140–239	Cancer and benign tumours
Other	Everything else	All other/unclassified codes

This grouping reduces the cardinality from thousands of codes to a fixed 9-category variable per diagnosis column. It also exploits the hierarchical structure of the ICD-9 system, where major code ranges correspond to physiological systems.

A separate binary flag, `primary_diag_is_diabetes`, is constructed to indicate whether the primary diagnosis (`diag_1`) falls in the diabetes category. This captures the clinically important distinction between patients admitted primarily *for* diabetes complications and those admitted for something else who happen to also have diabetes.

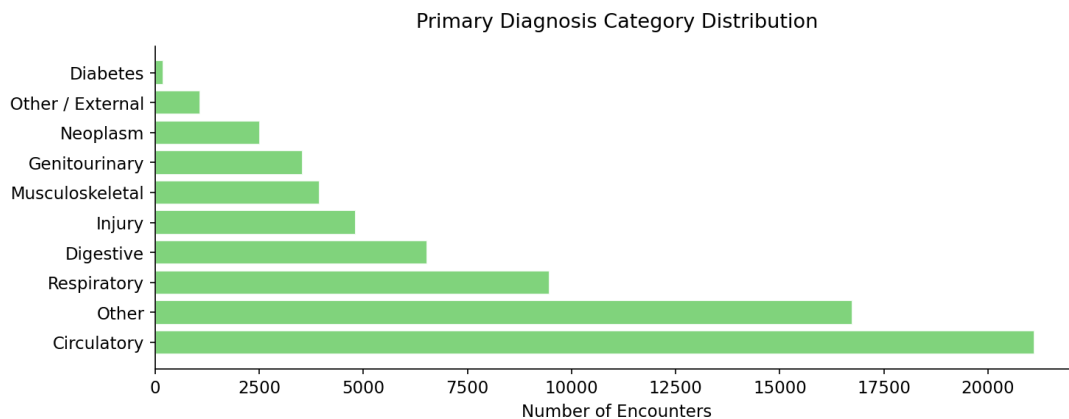


Figure 4.1. Distribution of primary diagnosis categories across the 69,987-encounter working cohort. Circulatory disease is the most common primary reason for admission ($\approx 35\%$), reflecting the strong cardiovascular comorbidity burden in diabetic populations. Diabetes-as-primary-diagnosis accounts for only $\approx 6\%$ of encounters, confirming that the vast majority of patients are admitted for complications related to, rather than directly caused by, their diabetes.

4.3. Age Ordinal Encoding

The `age` column contains decade brackets stored as strings: `[0–10)`, `[10–20)`, $\dots$, `[90–100)`. These are converted to ordinal integers 0–9 using a fixed mapping:

```

1 AGE_ORDER = {
2     '[0-10)': 0, '[10-20)': 1, '[20-30)': 2, '[30-40)': 3,
3     '[40-50)': 4, '[50-60)': 5, '[60-70)': 6, '[70-80)': 7,
4     '[80-90)': 8, '[90-100)': 9
5 }
6 df['age_ordinal'] = df['age'].map(AGE_ORDER)

```

Listing 4.1. Age ordinal encoding.

The ordinal encoding is preferred over one-hot encoding here because age is an ordered variable: being 70–80 is “more” than being 50–60 in the sense that older age is associated with higher readmission risk. One-hot encoding would discard this ordering, forcing the model to re-learn it from the data with fewer examples per age bracket. Ordinal encoding preserves the ordering while keeping dimensionality low.³

4.4. The 24 Medication Columns

The 24 medication change columns are encoded as ordered integers using the mapping No = 0, Steady = 1, Down = 2, Up = 3. This semi-ordinal encoding captures the idea that a dose change (either up or down) represents a more intensive medication management action than steady dosing, which in turn represents more engagement than no prescription.

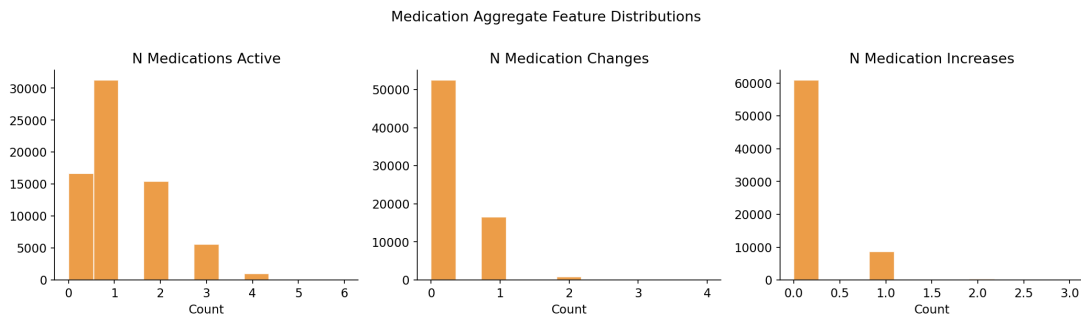


Figure 4.2. Distributions of the three aggregate medication features split by readmission status (0 = not readmitted, 1 = readmitted within 30 days). Readmitted patients have consistently higher mean values across all three aggregates, validating the clinical hypothesis that medication complexity is associated with readmission risk.

Three aggregate features are derived from the medication columns:

$$n_medications_active = \sum_{j=1}^{24} \mathbf{1}[med_j > 0] \quad (4.1)$$

³The choice between ordinal and one-hot encoding for ordered categories is discussed in Bishop (2006), Chapter 1. A practical decision framework is at <https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features>.

$$\text{n_medication_changes} = \sum_{j=1}^{24} \mathbf{1}[\text{med}_j \in \{2, 3\}] \quad (4.2)$$

$$\text{n_medication_increases} = \sum_{j=1}^{24} \mathbf{1}[\text{med}_j = 3] \quad (4.3)$$

The clinical hypothesis underlying these aggregates is that patients requiring many medication adjustments during their hospitalisation are more pharmacologically complex and, therefore, at higher risk of post-discharge complications if those adjustments are not well-managed at home.

4.5. Hospital Utilisation Features

Prior hospital utilisation is one of the strongest known predictors of future readmission in the literature. Two derived features capture this:

$$\text{total_prior_visits} = \text{number_outpatient} + \text{number_emergency} + \text{number_inpatient} \quad (4.4)$$

$$\text{had_prior_inpatient} = \mathbf{1}[\text{number_inpatient} > 0] \quad (4.5)$$

The continuous aggregate `total_prior_visits` captures overall healthcare engagement, while the binary flag `had_prior_inpatient` highlights the specific subset of patients who have already been hospitalised at least once before.

4.6. A1C Result and Glucose Serum Encoding

As noted in Chapter 3, we deliberately retained `A1Cresult` and `max_glu_serum` despite their high missingness rates, encoding them to preserve the clinical meaning of “test not ordered.”

For `A1Cresult`, the encoding is: `None` (not tested) $\rightarrow$ 0, `Norm` $\rightarrow$ 1, `>7` $\rightarrow$ 2, `>8` $\rightarrow$ 3.

A separate binary feature `A1C_tested` flags whether any A1C test was ordered ($= 1$ if result is not “None”, $= 0$ otherwise). Similarly, `glucose_tested` flags whether a serum glucose measurement was taken.

This dual-encoding strategy (ordinal level plus binary test flag) captures two distinct clinical signals: the *value* of the test when it was taken, and the *clinical decision* to order or not order the test.

4.7. Binary Clinical Flags

Three additional binary indicators are constructed:

$$\text{med_changed} = 1[\text{change} == \text{'Ch'}] \quad (4.6)$$

$$\text{on_diabetes_med} = 1[\text{diabetesMed} == \text{'Yes'}] \quad (4.7)$$

$$\text{primary_diag_is_diabetes} = 1[\text{diag_1_cat} == \text{'Diabetes'}] \quad (4.8)$$

where `change` is the raw column indicating whether any medication was changed (Ch) or not (No) during the encounter, and `diabetesMed` is the raw column indicating whether any diabetes medication was prescribed.

4.8. Gender Binary Encoding

The two valid gender values (Female, Male) are encoded as binary integers: Female = 0, Male = 1. This is a pragmatic encoding for binary logistic regression and tree-based models. We note the limitation of this representation for a clinical context: the dataset predates contemporary clinical approaches to gender diversity, and the binary encoding reflects the data as collected rather than a normative claim about gender.

4.9. Final Feature Space

After all encoding steps, the processed feature matrix has approximately **55–60 columns** (the exact count depends on the number of distinct category levels present in the specific train/val/test split). The features span a range of clinical domains:

- Utilisation and severity: `time_in_hospital`, `num_lab_procedures`, `num_medications`, `number_diagnoses`, `total_prior_visits`
- Demographics: `age_ordinal`, `gender_binary`, `race_*` (one-hot)
- Diagnosis categories: `diag_1_cat_*`, `diag_2_cat_*`, `diag_3_cat_*` (one-hot)
- Medication aggregates: `n_medications_active`, `n_medication_changes`, `n_medication_increases`
- Clinical flags: `med_changed`, `on_diabetes_med`, `had_prior_inpatient`, `A1C_tested`, `glucose_tested`
- Lab values: `A1Cresult_encoded`, `max_glu_serum_encoded`
- Individual medications: `insulin`, `metformin`, `glipizide`, etc. (24 columns, ordinal-encoded)

Figure 4.3 shows the distributions of the primary clinical features after engineering, split by readmission status.

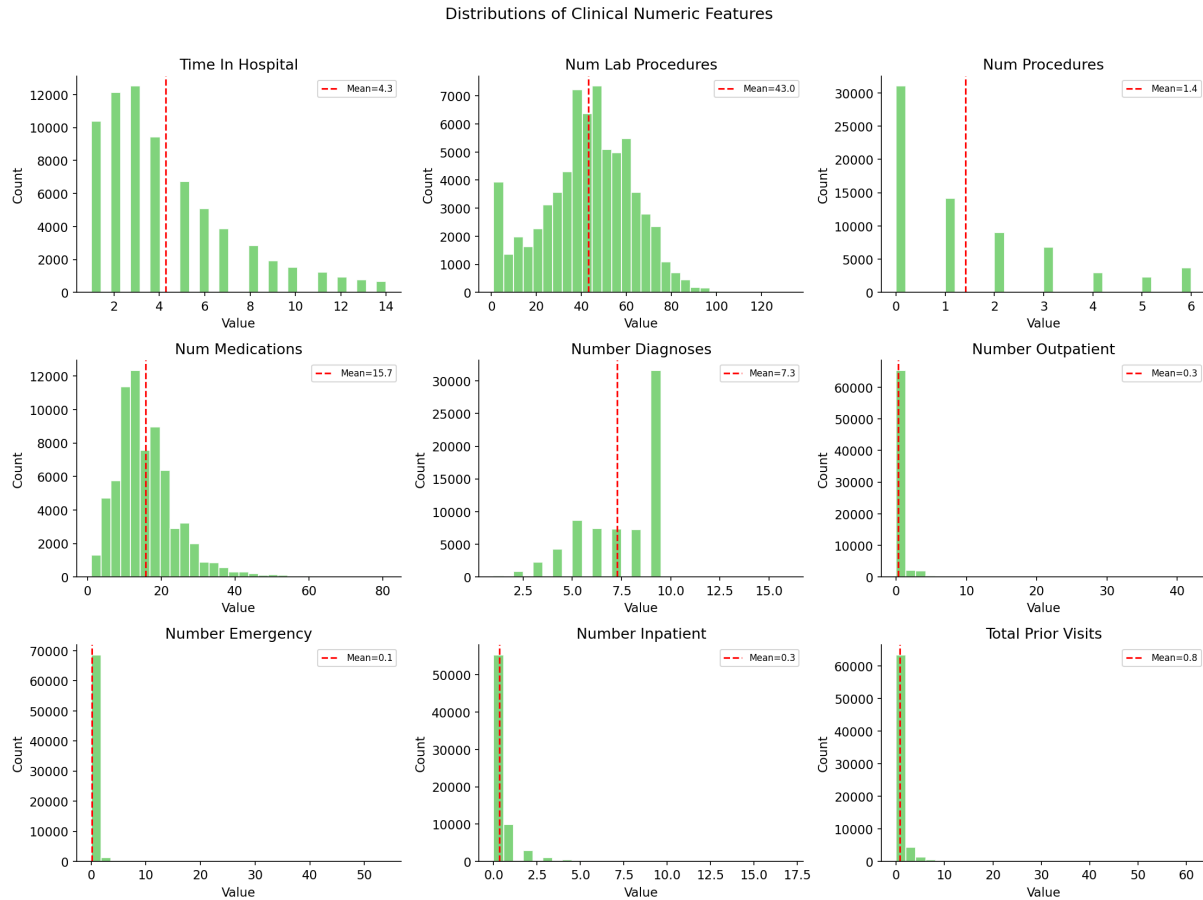


Figure 4.3. Histograms of the principal numeric clinical features in the processed diabetes dataset. The red dashed line in each histogram denotes the feature mean. Most utilisation-related variables exhibit strong right-skewness, indicating that while most patients have relatively few hospital visits, procedures, or medications, a small proportion experience substantially higher healthcare utilisation.

References

- Bishop, C. M. (2006). *Pattern recognition and machine learning*. New York, NY: Springer.
- Centers for Medicare & Medicaid Services. (2013). *Readmissions reduction program (HRRP)* (Tech. Rep.). U.S. Department of Health and Human Services. (Available at <https://www.cms.gov/medicare/payment/prospective-payment-systems/acute-inpatient-pps/readmissions-reduction-program-hrrp>)
- Donné, J., Aujesky, D., Williams, D., & Schnipper, J. L. (2013). Potentially avoidable 30-day hospital readmissions in medical patients: Derivation and validation of a prediction model. *JAMA Internal Medicine*, 173(8), 632–638. doi: 10.1001/jamainternmed.2013.3023
- Dua, D., & Graff, C. (2019). *UCI machine learning repository*. University of California, Irvine, School of Information and Computer Sciences. (Available at <https://archive.ics.uci.edu/ml>)
- Rubin, D. B. (1987). *Multiple imputation for nonresponse in surveys*. New York, NY: John Wiley & Sons.
- Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014, 781670. doi: 10.1155/2014/781670
- van Walraven, C., Bennett, C., Jennings, A., Austin, P. C., & Forster, A. J. (2011). Proportion of hospital readmissions deemed avoidable: A systematic review. *CMAJ*, 183(7), E391–E402. doi: 10.1503/cmaj.101860